

Refactoring the Security Module with a Level Contract Architecture

1. What is the "Level Contract" Principle?

In the context of Python modules, a **"level contract"** refers to enforcing a *layered architecture* or ordering among modules so that dependencies only flow in one direction (from higher-level components to lower-level ones). In simple terms, **higher-level modules can import lower-level modules, but not vice versa** ¹. Each module or package is assigned a *level* (or layer), and code at a given level **may depend only on the same level or lower levels**, never on higher levels. This forms a clear hierarchy of dependencies and prevents circular imports.

Such a level (layers) contract ensures that your code's dependencies form a **directed acyclic graph** – there are **no circular dependencies**, and all imports follow a top-down order ². A healthy architecture will have neatly organized dependency flow in one direction, which promotes *separation of concerns* and keeps each module focused on a single responsibility ². By contrast, without a level contract, large Python projects can devolve into a tangled web of imports, where modules are tightly coupled and frequently import each other, making the codebase hard to maintain ³.

In practice, enforcing a level contract often means introducing **abstract interfaces or service contracts** at the lower levels, which higher-level components rely on. This way, high-level logic does not directly depend on low-level implementations; instead, both depend on a stable abstraction. This concept is an application of the **Dependency Inversion Principle (DIP)** – *"High-level modules should not depend on low-level modules; both should depend on abstractions."* ⁴. By organizing code into layers and depending on abstract contracts, you can greatly reduce coupling and eliminate cyclic dependencies.

2. Refactoring Plan for `marty_msf.security` Package

The `marty_msf.security` module (part of the Marty Microservices Framework) currently contains a monolithic `unified_framework` module that has become a "god module" with very high coupling. According to the internal analysis, `security.unified_framework` has a coupling score of 16 (the highest in the project) and is imported by multiple other security components ⁵. This indicates that it contains numerous responsibilities and implementation details that others rely on, leading to tight interdependencies. The analysis explicitly flags it as an architectural problem: "`security.unified_framework` [is a] God Module with coupling score of 16" causing components to depend on implementation details rather than clean interfaces ⁶.

Our goal is to restructure the `marty_msf.security` package using the **level contract (layered)** approach. We will break up `unified_framework` into smaller, well-defined modules organized by responsibility and introduce an *internal layering* that decouples these pieces. The high-level plan is:

- **Split the `unified_framework` "god module" into focused submodules** – for example: `security.auth`, `security.authz`, `security.secrets`, and `security.api` (interfaces). Each will handle a distinct aspect of security.
- **Isolate shared interfaces in `security.api`** – define abstract contracts (e.g. `IAuthenticator`, `IAuthorizer`, `ISecretManager`) that describe the behaviors of the security components. This module will be the **lowest level** in our security package hierarchy (the foundation). It contains no business logic, only interface definitions and perhaps simple data types or exceptions. All other security submodules will depend on this `security.api` layer.
- **Refactor concrete implementations to depend only on `security.api` (and standard library)** – the `security.auth`, `security.authz`, and `security.secrets` modules will provide implementations of the interfaces. They **should not import each other directly**; any interaction must go through the interfaces in `security.api`. This ensures these components remain loosely coupled (fulfilling the level contract: they only depend on the lower-level interface layer, not on peer concrete modules).
- **Add a bootstrap (composition) module to wire everything together** – since the project may not be using an inversion-of-control container or a central composition root, we will create a simple `security.bootstrap` (or integrate into an existing startup module) that **instantiates and connects the components**. This is where dependencies are injected manually: we create the concrete classes and supply them with any required collaborators (via the interfaces). This bootstrap code is the only place that knows about all the implementations, keeping the rest of the codebase ignorant of the concrete classes and thus adhering to the Dependency Inversion Principle.

Following these steps will implement the recommendations from the analysis (which suggested extracting an `interfaces` module and splitting auth vs authz logic) ⁷. Below is a detailed breakdown of each part of the refactoring plan:

2.1 Splitting `unified_framework` into Focused Submodules

Firstly, we divide the responsibilities of `unified_framework` into separate modules/layers:

- `marty_msf.security.api` – **Interfaces and data contracts**. This will contain protocol or abstract base class (ABC) definitions for the core behaviors: for example, an `IAuthenticator` interface for authentication, an `IAuthorizer` for authorization decisions, and perhaps an `ISecretManager` for secret management. This module will have **no knowledge of any implementations**. It forms the lowest level of our security layer, meaning nothing in `security` depends on code within `api`. (It may also include simple shared types like a `User` data class or exceptions like `AuthenticationError`, if multiple components use them, though these could also live in a small `security.models` or `security.common` if preferred.)
- `marty_msf.security.auth` – **Authentication logic**. This module (or package) contains concrete authenticators that implement `IAuthenticator`. For example, you might have a `BasicAuthenticator` that verifies username/password against a user store, or classes that integrate with external identity providers (OIDC, SAML, etc.). The key is that `security.auth`

should import only from `security.api`, not from `security.authz` or other siblings. If it needs to perform any authorization or use any secrets, it will rely on interfaces defined in `security.api` (e.g., call an `IAuthorizer` or `ISecretManager` passed in) rather than directly import another module.

- `marty_msf.security.authz` - **Authorization logic.** This contains implementations of `IAuthorizer` for permission checks. For instance, a `RoleAuthorizer` that checks user roles/permissions, or policy engine integrations (ABAC, RBAC, ACL engines, etc.). Like `auth`, this module only depends on `security.api`. If an authorizer needs data from authentication or secrets, it should be provided via an interface. (For example, an authorizer could use a user attribute or fetch additional context via an abstraction, rather than importing the authenticator or calling it directly.)
- `marty_msf.security.secrets` - **Secret management and utilities.** This module implements `ISecretManager` or similar interfaces if there are shared secret operations (for example, key storage, password hashing salt, token encryption keys, etc.). Concrete classes here might include a `EnvSecretManager` (fetching secrets from environment or config) or a `VaultSecretManager` (integrating with an external secret store). Again, this module depends only on `security.api`. Other parts of security that need secrets (e.g., an authenticator needing an encryption key to verify a token) would accept an `ISecretManager` from this layer, rather than importing `security.secrets` directly.
- (Optional) `marty_msf.security.core` or `security.common` - since the user specifically mentioned `auth`, `authz`, and `secrets`, we likely won't need a separate `core` module beyond the `api`. However, if there is any non-interface shared functionality (like common utilities or domain classes not suitable for the interface layer), a small `core/common` module can house them. This module would be treated like a lower layer as well (likely alongside `api`). In many cases, though, you can avoid this by keeping shared things either abstract (in `api`) or at a higher level (wired in bootstrap).

By splitting into these parts, each piece has a clear responsibility. For example, all authentication-related code is in `security.auth`, completely separated from authorization code in `security.authz`. They communicate only through defined interfaces if needed. This adheres to the **Single Responsibility Principle** and makes the code easier to reason about. It also sets up our level contract: `security.api` is the foundational layer, and `auth`, `authz`, `secrets` are higher-level implementation layers that must not depend on each other (only on `api`). The original `security.unified_framework` was doing too much and importing everything; after refactoring, its functionality will be distributed and the module can be removed or reduced to a facade if absolutely needed.

2.2 Defining Shared Interfaces in `security.api`

Next, we create the **interfaces** that will serve as the abstraction contracts between components. In Python, we can use **protocols** (from the `typing` module) or **abstract base classes** (`abc.ABC`) to define these. The choice depends on preference; protocols allow structural subtyping (classes just need to have the right methods to be considered a match) while ABCs enforce subclassing and implementation of abstract methods. For simplicity and flexibility, let's use **protocols** in this example.

In `marty_msf/security/api.py` (or as an `api` subpackage with an `__init__.py`), we define interfaces like so:

Interface Declaration (`security.api`):

```
from typing import Protocol, runtime_checkable
from dataclasses import dataclass

@dataclass
class User:
    """Example User data model (could be more complex in real code)."""
    username: str
    roles: list[str]

@runtime_checkable
class IAuthenticator(Protocol):
    """Interface for authentication providers."""
    def authenticate(self, username: str, password: str) -> User:
        """Authenticate credentials and return a User, or raise exception if
        invalid."""
        ...

@runtime_checkable
class IAuthorizer(Protocol):
    """Interface for authorization providers."""
    def authorize(self, user: User, action: str) -> bool:
        """Check if the given user is allowed to perform the specified
        action."""
        ...

@runtime_checkable
class ISecretManager(Protocol):
    """Interface for secret management (e.g., retrieving keys or secrets)."""
    def get_secret(self, key: str) -> str:
        """Retrieve a secret value by key (could be from env, vault, etc.)."""
        ...
```

In the above snippet, we define a simple `User` data class used by the interfaces (for authentication and authorization). The `IAuthenticator` Protocol declares an `authenticate` method, the `IAuthorizer` declares an `authorize` method, and `ISecretManager` declares a method to fetch secrets. The use of `Protocol` (with `@runtime_checkable` for optional runtime `isinstance` checks) means that any class with the required methods will be considered as implementing the interface, even if it doesn't explicitly inherit from `IAuthenticator` or so. We could alternatively use ABCs, e.g., `class IAuthenticator(ABC)` with `@abstractmethod` definitions, which would require subclassing to implement. Either way, the idea is that **these interfaces carry no concrete logic or dependencies** – they don't import anything from `security.auth/authz/secrets`. This module is self-contained

(depending only on Python stdlib like `typing` and perhaps shared simple classes like `User`). By having consumers depend on these interfaces, we ensure the higher-level modules “**depend on abstractions**” rather than concrete implementations ⁴.

2.3 Implementations in `auth`, `authz`, and `secrets` (Depending Only on `security.api`)

With interfaces in place, we refactor the contents of `unified_framework` into the new implementation modules. Each implementation will import the necessary interface(s) from `marty_msf.security.api` and implement their methods. **They will not import or directly call code from the other implementation modules.** Any interaction goes through an interface.

For example, suppose in the old design the unified framework did both authentication and authorization. Now we might have:

- In `marty_msf.security.auth`: one or more classes implementing `IAuthenticator`. Let's illustrate a simple one:

Concrete Implementation (`security.auth`):

```
# file: marty_msf/security/auth.py
from marty_msf.security.api import IAuthenticator, ISecretManager, User

class BasicAuthenticator:
    """Simple authenticator that verifies username/password against a stored
    secret."""
    def __init__(self, secret_manager: ISecretManager):
        # The authenticator depends on an abstract secret manager (for, say,
        # password hashing salt or token key).
        self._secret_manager = secret_manager

    # This method signature matches IAuthenticator.authenticate
    def authenticate(self, username: str, password: str) -> User:
        # For illustration: retrieve the expected password or hash from secret
        # store:
        key = f"user.{username}.password"
        expected_password = self._secret_manager.get_secret(key)
        if expected_password is None:
            raise ValueError("Unknown user")
        if password == expected_password: # In real life, hash and compare!
            # Authentication successful; return a User object (with roles
            # perhaps fetched elsewhere)
            return User(username=username, roles=[])
        else:
            raise ValueError("Invalid credentials")
```

In this `BasicAuthenticator` example, notice a few things: - It does **not** import anything from `security.authz` or other modules. If it needs to perform an authorization step as part of login (not typical, but suppose it did), it should accept an `IAuthorizer` in its constructor or method parameter rather than directly importing `security.authz`. (In most cases, authentication and authorization are separate steps, so they might not directly call each other at all – which is even simpler.) - It **depends on** `ISecretManager` **from** `security.api` for retrieving secrets. Here we simulate fetching a stored password. The authenticator doesn't know how secrets are stored; it just calls `get_secret`. This is dependency inversion in action – it relies on an abstraction. The actual `SecretManager` implementation will be injected later (in bootstrap). This way, `BasicAuthenticator` is decoupled from any specific secret storage mechanism (it could work with an env-based secret manager, a vault-based one, etc., as long as it meets the interface).

- In `marty_msf.security.authz`: one or more classes implementing `IAuthorizer`. For example:

```
# file: marty_msf/security/authz.py
from marty_msf.security.api import IAuthorizer, User

class RoleAuthorizer:
    """Authorizer that grants access based on user roles."""
    def authorize(self, user: User, action: str) -> bool:
        # Simple example: allow if user's roles contain the action name.
        # In reality, this could check a permissions mapping, policy engine,
        etc.
        return action in user.roles
```

This `RoleAuthorizer` implements the `authorize` method defined by `IAuthorizer`. It only imports the `User` class and interface type from `security.api`. It does not import the authenticator or anything else. If a more complex authorizer needed, say, to query user information or other services, it should ideally be given those dependencies via interfaces as well.

- In `marty_msf.security.secrets`: classes implementing `ISecretManager`. For example, a trivial implementation that reads from an environment variable or a dictionary:

```
# file: marty_msf/security/secrets.py
import os
from marty_msf.security.api import ISecretManager

class EnvSecretManager:
    """Secret manager that retrieves secrets from environment variables."""
    def get_secret(self, key: str) -> str:
        # Simply use an env var named after the key (for demo purposes)
        return os.getenv(key)
```

Again, this class depends only on `security.api` and standard library (`os`). We could have other implementations, like `HardcodedSecretManager` (for testing), `VaultSecretManager` (for a HashiCorp Vault integration), etc., without changing the authenticator code – because the authenticator only cares about the `ISecretManager` interface.

Now each of these modules (`auth.py`, `authz.py`, `secrets.py`) is at the *same layer* in our architecture: they implement the business logic, and all three depend on the lower-layer `api` definitions. They **do not depend on each other**. This modularizes the security package. For instance, you can modify how secrets are stored or how authorization is done without touching the authentication module, as long as the interface contracts remain the same.

2.4 Wiring Components Together in a Bootstrap Module

After splitting the code, the remaining question is: *How do these pieces come together at runtime?* Without a dependency injection framework, we will **wire the components manually** in a designated place. This is often called the **composition root** of the application – a central place where you assemble the object graph. In our case, we can create a `marty_msf.security.bootstrap` module (or integrate this into the application startup code) that knows about the concrete classes and composes them.

For example, `marty_msf/security/bootstrap.py` might contain something like:

Wiring in Bootstrap (`security.bootstrap`):

```
# file: marty_msf/security/bootstrap.py
from marty_msf.security.auth import BasicAuthenticator
from marty_msf.security.authz import RoleAuthorizer
from marty_msf.security.secrets import EnvSecretManager
from marty_msf.security.api import IAuthenticator, IAuthorizer, ISecretManager

def init_security_components():
    """Initialize and wire up security components."""
    # Instantiate the concrete implementations
    secret_manager: ISecretManager = EnvSecretManager()
    authorizer: IAuthorizer = RoleAuthorizer()
    authenticator: IAuthenticator =
    BasicAuthenticator(secret_manager=secret_manager)
    # If BasicAuthenticator also needed an authorizer (not in our simple case),
    we could pass it in similarly.
    return authenticator, authorizer, secret_manager

# Example usage:
if __name__ == "__main__":
    auth, authz, secrets = init_security_components()
    user = auth.authenticate("alice", "s3cr3tpass")
    if authz.authorize(user, "create_order"):
        print(f"{user.username} is authorized to create_order")
```

```

else:
    print(f"Access denied for {user.username}")

```

A few notes on this approach: - We explicitly import the concrete classes here in the bootstrap. This is acceptable because the bootstrap is *meant* to know about all layers. It's the composition root that ties interfaces to implementations. The rest of the application, however, could just import `IA Authenticator` or use the returned objects without directly importing (or knowing about) `BasicAuthenticator` or `EnvSecretManager` classes. - We instantiate an `EnvSecretManager`, then a `RoleAuthorizer`, then a `BasicAuthenticator` providing it with the `secret_manager`. At this point, all dependencies are satisfied. We could then provide these objects to whatever part of the system needs them (for example, the web application might use `authenticator` to verify login credentials, and use `authorizer` to check permissions). - Even without a fancy DI container, this manual approach achieves **dependency injection**. We inject the `secret_manager` into the authenticator by passing it in. We could just as easily swap in a different `ISecretManager` implementation here (for example, for tests or different environments) without changing the authenticator code. This flexibility is a key advantage of the layered approach. - The `bootstrap.py` essentially plays the role of the DI composition root. In larger applications, you might call `init_security_components()` during application startup to get your security-related objects configured. As noted in *Cosmic Python*, having a separate bootstrap is a clean way to handle wiring; it centralizes object construction in one place (the *composition root*) ⁸.

By following this plan (splitting modules, introducing interfaces, and wiring in one place), we significantly reduce inter-module coupling. Each part of the security subsystem deals with its own concern and relies on an abstraction for anything outside its scope. This makes the codebase more maintainable and testable (for example, you can unit test the authenticator by passing in a stub `ISecretManager` without pulling in the real secret store).

3. How the Level Contract Prevents Circular Dependencies

Adopting the level contract architecture in the `marty_msf.security` package effectively **eliminates circular dependencies** by construction. Here's why:

- **One-way dependency flow:** We established a clear ordering: `security.api` (interfaces) is the lowest level, and the other modules (`auth`, `authz`, `secrets`) are on a higher level that depend on this interface layer. The implementation modules do **not** depend on each other. Any would-be cross-dependency is routed through `security.api`. For example, if `auth` needs something that `secrets` provides, it doesn't import `secrets` directly; it uses `ISecretManager`, which is defined in the lower layer. This means there is no scenario where `auth` imports `secrets` *and* `secrets` imports `auth` (which would create a cycle) – instead, both import `api` and are agnostic of each other's internals.
- **No mutual imports at the same level:** The authenticator and authorizer implementations know about each other only via abstract protocols, if at all. In our example, they actually don't reference each other at all (which is ideal). Even if there is some interaction (say an authenticator wants to check a permission during login), it would call an interface method on an injected authorizer, not import the authorizer module. This decoupling means you won't have import cycles like the original code did. The original `unified_framework` was importing many things (13 imports) and also

being imported by others ⁹, which easily could form cycles. After refactoring, `security.api` is imported by others but it itself imports nothing from them, and the siblings (`auth`, `authz`, etc.) don't import each other – so cycles are impossible unless someone violates the architecture.

- **Enforcing separation of concerns:** Each module has a single responsibility (auth vs authz vs secret handling). This separation means there's less temptation to cross-call in tangled ways. If one module grows to need capabilities of another, that's a sign those capabilities should be abstracted to the `api` layer (or perhaps moved to a common utility) so that the dependency can remain one-way. The level contract essentially guides developers to put shared stuff in the lower layer and implementation in upper layers, which naturally avoids cyclical relationships. All dependency arrows point downward to the `api` layer (or to external libraries), never sideways or upward – keeping the dependency graph acyclic ².
- **Dependency Inversion Principle (DIP):** As mentioned, by depending on abstractions, we invert the typical dependency direction. High-level policy (e.g., “how do we authenticate?”) is defined by an interface, and low-level details (e.g., “use env vars for secrets” or “check these roles”) implement that. Both high-level and low-level modules depend on the interface definition, not on each other. This **prevents circular dependency** because the “link” between modules is an interface that does not import either implementation. In other words, instead of $A \rightarrow B$ directly, and $B \rightarrow A$ (which would form a cycle), we have $A \rightarrow \text{Interface}$, $B \rightarrow \text{Interface}$, and at runtime A and B are connected through that interface by the bootstrap. There is no direct $A \rightarrow B$ import to cause a cycle.

Overall, once this refactoring is applied, the architecture will have **all dependencies flowing in one direction (toward the `security.api` layer)** and no circular references. This not only fixes the current circular import issues but also makes future development safer: if someone inadvertently tries to import, say, `security.auth` from `security.secrets`, it would be clearly against the defined layering. By keeping the contract, developers are forced to think “*Do I really need something from that module? Or should this interaction be done through an interface?*” – which leads to better design decisions and maintainable code. As an added benefit, having no cycles and clear separations will make the code easier to test and understand. New developers can see the layer structure and know, for example, that `auth` and `authz` communicate via the API layer, not through hidden imports. This reduces the “spaghetti” effect that was earlier noted as a problem.

4. Enforcing the Level Contract with Import Linter (Pre-Commit Check)

To ensure that the **level contract** remains intact going forward, we can introduce automated checks. One useful tool for this is **Import Linter** (`import-linter`), which allows you to define architectural contracts about module dependencies and verifies them. We can set up an **Import Linter layers contract** specifically for the `marty_msf.security` package.

For example, in an `.importlinter` config file, we could add a contract like:

```
[importlinter:contract:security_layers]
name = Enforce layered architecture in security
```

```

type = layers
containers = marty_msf.security
layers =
    auth | authz | secrets
    api

```

Let's break down what this means: - We treat `marty_msf.security` as the container (so we don't have to repeat the full package name for each layer). - In the `layers` list, we specified two levels: - The top layer: `auth | authz | secrets` - separated by `|` which means these are **independent siblings on the same layer** ¹⁰. Import Linter will consider `auth`, `authz`, and `secrets` as peers that **must not import from each other**. (Using the pipe `|` indicates they are independent; none of them can depend on the others, which is exactly what we want.) - The bottom layer: `api` - this is below the others. The layers contract will enforce that none of the top-layer modules import from the `api` layer? Actually, it's the opposite: *higher-level modules can import lower-level ones, but not vice versa* ¹. By listing `api` last, we declare it as the lowest layer. This means: - `security.api` **cannot** import anything from `auth`, `authz`, or `secrets` (which is correct - our interface module should be self-contained). - `security.auth`, `security.authz`, and `security.secrets` are allowed to import from `security.api` (since `api` is below them), but **not from each other** (enforced by being siblings) and not from any higher layer (there is none higher in this contract). - With this contract, if someone accidentally writes `from marty_msf.security.auth import BasicAuthenticator` inside `secrets.py`, or any other forbidden import, the linter will catch it as a broken contract.

We can integrate this check into a **pre-commit hook or CI pipeline**. For instance, using **pre-commit**, we can add a hook that runs `lint-imports` (the command for Import Linter). In `.pre-commit-config.yaml` it might look like:

```

- repo: local
  hooks:
    - id: import-linter-security
      name: Import Linter (Security Layer Contract)
      entry: lint-imports
      language: system # or python
      pass_filenames: false

```

This assumes the `.importlinter` file is present at the root of the repository. When a commit is attempted, the hook will run `lint-imports`. If any import in `marty_msf.security` violates our defined layers contract, `lint-imports` will exit with an error, causing the commit to fail. This immediately alerts the developer that they are introducing an illegal dependency. Similarly, in a CI pipeline, you can run `lint-imports` as part of the tests. (The Import Linter documentation suggests adding it to your test pipeline to ensure the team adheres to the architectural rules ¹¹.)

Because we specifically crafted the contract to mirror our intended architecture, the linter serves as a **guardian of the level contract**. It will prevent regressions like someone accidentally re-introducing a circular import. For example, if a future change tries to make `security.api` import something from `security.auth` (which should never happen), the linter would flag it since `api` is a lower layer and

cannot import higher layers. Or if `security.authz` tried to import a class from `security.auth` directly, that would break the sibling independence rule ¹⁰.

In addition to Import Linter, another approach could be using **Importlab** or **custom scripts**, but Import Linter's declarative contract is quite convenient. There are also similar tools like **Layer Linter** or even writing a custom **flake8** plugin for imports. However, Import Linter is designed for exactly this scenario and supports the layers pattern out of the box.

Summary: We define a layers contract for the security package (auth, authz, secrets can depend on api, but not on each other), and then automate checks so that any violation is caught during development (pre-commit) or at least in continuous integration. This ensures the **level contract architecture remains enforced** as the project grows, preventing accidental introduction of circular dependencies or inappropriate coupling in the security module. By keeping this contract, the security code will stay clean, modular, and maintainable, matching the intended design moving forward.

Sources:

- Piglei Blog – “higher-level modules can freely import content from lower-level modules, but not vice versa.” ¹
- Piglei Blog – One-directional dependencies enable separation of concerns and no circular imports ²
- Project Architecture Report – `security.unified_framework` identified as a highly coupled *God Module* (coupling score 16) ⁶ ⁵
- Project Architecture Plan – Recommendation to split `unified_framework` and introduce `security.interfaces` (API) layer ⁷
- Piglei Blog – Using Protocols for DIP: “introducing a Protocol to decouple... is an application of the Dependency Inversion Principle... high-level modules and low-level modules both depend on abstractions.” ⁴
- Cosmic Python – On using a bootstrap (composition root) to wire dependencies in one place ⁸
- Import Linter Docs – Siblings in same layer (separated by `|`) are independent (cannot import each other) ¹⁰
- David Seddon – Suggestion to use `lint-imports` in CI for architectural rules enforcement ¹¹

¹ ² ³ ⁴ 6 ways to improve the architecture of your Python project (using import-linter) | Piglei
<https://www.piglei.com/articles/en-6-ways-to-improve-the-arch-of-you-py-project/>

⁵ ⁶ ⁷ ARCHITECTURE_ANALYSIS_REPORT.md
file:///file_00000000c8e8622fa179704f0b670917

⁸ Dependency Injection (and Bootstrapping)
https://www.cosmicpython.com/book/chapter_13_dependency_injection.html

⁹ internal_import_analysis.json
file:///file_00000000dbb461f58387ceaf44be6fcb

¹⁰ Contract types — Import Linter 2.5.2 documentation
https://import-linter.readthedocs.io/en/stable/contract_types.html

11 Meet Import Linter · David Seddon

<https://seddonym.me/2019/05/20/meet-import-linter/>